

K vecinos más cercanos (K-NN): el laboratorio del curso, reproducido y extendido

Laboratorio de Python paso a paso · Machine Learning · UTB 2026-II

Referencia de estudio basada en la clase y el laboratorio de K-NN

2026-10-09

Tabla de contenidos

1	Importar las librerías	2
2	Importar los datos	3
3	Partir los datos: las dos formas	4
3.1	Forma 1 · Determinista (la del profesor)	4
3.2	Forma 2 · Aleatoria (la suya)	5
4	Escalar las variables	5
5	La regla de decisión (la fórmula del curso)	5
6	Entrenar el K-NN	6
7	Predecir un cliente nuevo	6
7.1	Las dos vías: a mano y con la librería	6
8	Predecir el conjunto de prueba	7
9	Las métricas	7
10	Ver la frontera del modelo	9
11	K-NN a mano: el ejercicio que puede caer en el parcial	9
11.1	Paso a paso con un solicitante	10
11.2	El efecto de k sobre la votación	11
12	El parámetro que lo cambia todo: k	12
12.1	De memorizar a aplanar	12
13	La trampa de K-NN	13
13.1	Si no escala, una variable se come a la otra	13
13.2	El experimento de la clase: la misma información, otras unidades	13
13.3	Las dos debilidades que hay que decir en voz alta	14
14	Tres ideas para llevarse	14
15	Resumen consolidado	14

i Lo que vamos a hacer hoy

El mismo recorrido del laboratorio de clase, sobre los **400 clientes de datos/cartera_curso.csv**: cargar, partir, escalar, entrenar, predecir, medir y **dibujar**.

Con la diferencia que es el corazón de la sesión: la partición se hace de **dos maneras** (la del profesor, determinista; la suya, aleatoria con su semilla) y al final **se comparan los dos resultados**.

Y un extra de repaso para el parcial: el **K-NN a mano** con 10 clientes — incluidos los dos casos donde escalar **cambia la predicción** — y los dos experimentos de la clase de teoría (las unidades y la maldición de la dimensión).

Métricas de hoy: matriz de confusión, exactitud, **tasa base**, precisión, recall y F1. (ROC y AUC vienen en la clase de evaluación.)

1. Importar las librerías

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from IPython.display import Markdown, display

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (confusion_matrix, accuracy_score,
                             precision_score, recall_score, f1_score)

plt.rcParams.update({"figure.dpi": 130, "font.size": 9,
                    "axes.spines.top": False, "axes.spines.right": False})
ROJO, VERDE, AZUL, NARANJA, GRIS = ("D81B60", "009E73", "0072B2",
                                     "E69F00", "999999")

def texto(md):
    """Imprime Markdown enriquecido en el output (la 'lectura' de cada resultado)."""
    display(Markdown(md))

def tabla(df, decimales=3):
    """Convierte un DataFrame en tabla Markdown con columnas centradas
    (la utilidad de salida del curso). Redondea las columnas float."""
    d = df.copy()
    for c in d.columns:
        if pd.api.types.is_float_dtype(d[c]):
            d[c] = d[c].map(lambda v: f"{v:.{decimales}f}")
    cab = "|" + " | ".join(str(c) for c in d.columns) + " |"
    sep = "|" + "|".join([":--:"] * len(d.columns)) + "|"
    fil = ["| " + " | ".join(str(v) for v in r) + " |" for r in d.values]
    return Markdown("\n".join([cab, sep] + fil))
```

2. Importar los datos

```
# CAMBIA AQUÍ la ruta si mueve el archivo o va a RStudio Cloud.
# (En el laboratorio de clase el archivo se llamó datos_knn_clase.csv;
# aquí usamos la misma cartera del curso.)

datos = pd.read_csv("datos/cartera_curso.csv", sep=";", decimal=",")
print(datos.shape)
datos.head()
```

(400, 4)

	id_cliente	edad	ingreso	tomo_credito
0	C125	54	7.6	1
1	C037	26	6.5	0
2	C298	20	2.8	0
3	C339	45	7.3	1
4	C050	34	7.6	1

```
print(datos.dtypes, "\n")
print(datos[["edad", "ingreso"]].describe().round(2), "\n")
print("¿cuántos tomaron el crédito?")
print(datos["tomo_credito"].value_counts(), "\n")
print("proporción que SÍ tomó:", round(datos["tomo_credito"].mean(), 3))
```

```
id_cliente      str
edad            int64
ingreso         float64
tomo_credito    int64
dtype: object
```

```
      edad  ingreso
count  400.00  400.00
mean    39.18    5.32
std     11.89    2.46
min     19.00    1.00
25%     29.75    3.08
50%     38.50    5.40
75%     49.00    7.40
max     60.00    9.50
```

```
¿cuántos tomaron el crédito?
tomo_credito
1      216
0      184
Name: count, dtype: int64
```

```
proporción que SÍ tomó: 0.54
```

Qué mirar: que edad e ingreso lleguen como números (si dicen `object`, el archivo se leyó con el separador equivocado), y **cuál es la clase mayoritaria:** ese porcentaje es la **tasa base**, el número que nuestro modelo tiene que superar. Nota también los rangos: edad de 19 a 60 años, ingreso de 1 a 9.5 millones — escalas distintas, y eso es exactamente lo que arregla el escalado.

```
fig, ax = plt.subplots(figsize=(6.5, 4))
for cls, color, etq in [(0, ROJO, "no tomó"), (1, VERDE, "sí tomó")]:
    d = datos[datos["tomo_credito"] == cls]
    ax.scatter(d["edad"], d["ingreso"], c=color, s=18, alpha=0.7,
              edgecolor="white", linewidth=0.3, label=etq)
ax.set_xlabel("edad"); ax.set_ylabel("ingreso (millones)")
ax.legend(frameon=False)
plt.tight_layout(); plt.show()
```

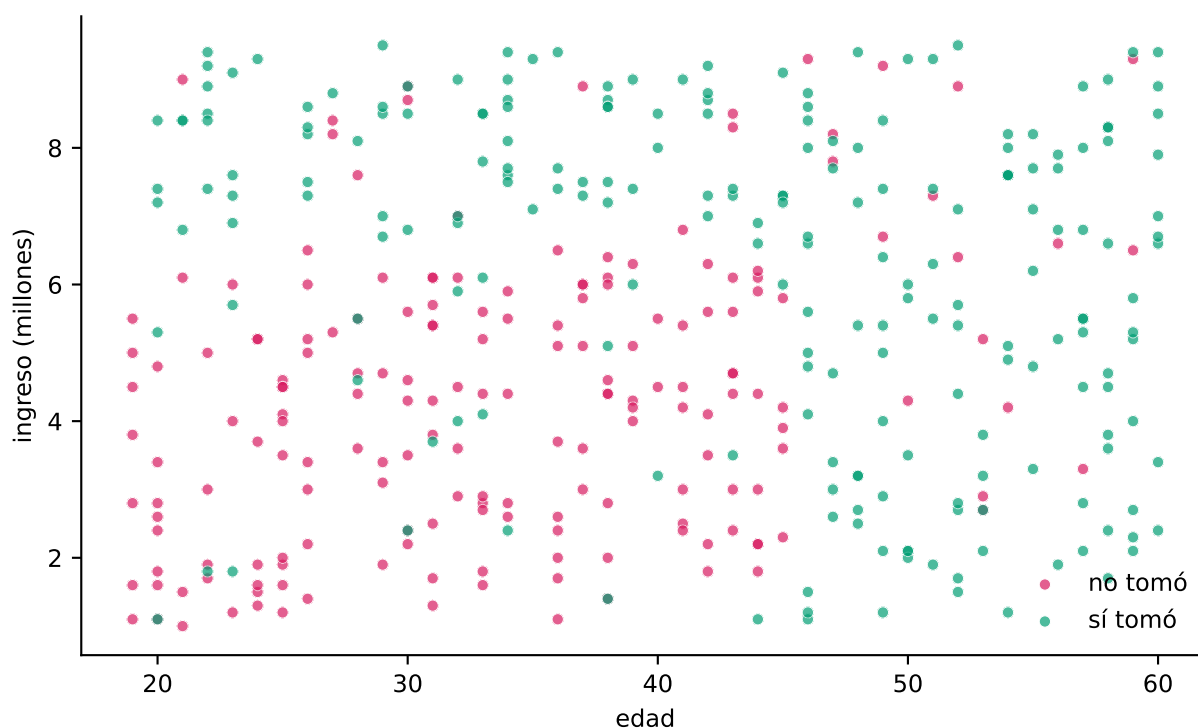


Figura 1: Los 400 clientes. Verde = tomó el crédito.

3. Partir los datos: las dos formas

3.1. Forma 1 · Determinista (la del profesor)

Se ordena por el código del cliente y se toma el **primer 75 %**. Siempre da lo mismo, en cualquier computador.

```
d = datos.sort_values("id_cliente").reset_index(drop=True)
n_train = int(np.floor(0.75 * len(d)))

X_train_d = d.loc[:n_train-1, ["edad", "ingreso"]].values
y_train_d = d.loc[:n_train-1, "tomo_credito"].values
```

```
X_test_d = d.loc[n_train:, ["edad", "ingreso"]].values
y_test_d = d.loc[n_train:, "tomo_credito"].values

print(f"entrenamiento: {len(X_train_d)} · prueba: {len(X_test_d)}")
```

entrenamiento: 300 · prueba: 100

3.2. Forma 2 · Aleatoria (la suya)

```
MI_SEMILLA = 0          # <<< CAMBIE ESTE NÚMERO POR LOS DOS ÚLTIMOS DÍGITOS DE SU CÓDIGO

X = datos[["edad", "ingreso"]].values
y = datos["tomo_credito"].values
X_train_a, X_test_a, y_train_a, y_test_a = train_test_split(
    X, y, test_size=0.25, random_state=MI_SEMILLA)

print(f"entrenamiento: {len(X_train_a)} · prueba: {len(X_test_a)}")
```

entrenamiento: 300 · prueba: 100

4. Escalar las variables

La edad va de 19 a 60 y el ingreso de 1 a 9.5: **sin escalar, la edad decide sola quién es vecino.**

```
sc_d = StandardScaler().fit(X_train_d)          # ojo: fit SOLO con entrenamiento
X_train_d_s, X_test_d_s = sc_d.transform(X_train_d), sc_d.transform(X_test_d)

sc_a = StandardScaler().fit(X_train_a)
X_train_a_s, X_test_a_s = sc_a.transform(X_train_a), sc_a.transform(X_test_a)

print("antes de escalar (primer cliente):", X_train_d[0])
print("después de escalar           :", X_train_d_s[0].round(3))
```

antes de escalar (primer cliente): [45. 3.9]
 después de escalar : [0.57 -0.543]

5. La regla de decisión (la fórmula del curso)

Sea $N_k(x)$ el conjunto de los k vecinos más cercanos a x en el entrenamiento. La probabilidad estimada y la clase son:

$$\hat{P}(y = 1 \mid x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i \quad \hat{y} = \begin{cases} 1 & \text{si } \hat{P} \geq 0.5 \\ 0 & \text{si no} \end{cases}$$

Como las etiquetas son 0 y 1, ese promedio **es** la proporción de vecinos que tomaron el crédito — la votación del dibujo de la clase. Con k par puede haber empate exacto; por eso el curso prefiere k **impar** (y scikit-learn, en caso de empate, se queda con la clase 0).

6. Entrenar el K-NN

```
K = 9 # <<< CAMBIA AQUÍ el número de vecinos

modelo_d = KNeighborsClassifier(n_neighbors=K, metric="minkowski", p=2)
modelo_d.fit(X_train_d_s, y_train_d)

modelo_a = KNeighborsClassifier(n_neighbors=K, metric="minkowski", p=2)
modelo_a.fit(X_train_a_s, y_train_a)

print("modelos entrenados con k =", K)
```

modelos entrenados con k = 9

7. Predecir un cliente nuevo

```
nuevo = [[38, 7.2]] # <<< CAMBIA AQUÍ: 38 años, 7.2 millones de ingreso
p = modelo_d.predict_proba(sc_d.transform(nuevo))[0, 1]
print(f"probabilidad de que tome el crédito: {p:.3f}")
print("clase predicha:", modelo_d.predict(sc_d.transform(nuevo))[0])
print(f"(de los {K} vecinos más parecidos, {int(round(p*K))} tomaron el crédito)")
```

probabilidad de que tome el crédito: 0.667
clase predicha: 1
(de los 9 vecinos más parecidos, 6 tomaron el crédito)

7.1. Las dos vías: a mano y con la librería

El criterio del curso: **todo lo que hace la librería se puede reproducir a mano**, y en K-NN son tres líneas de numpy.

```
def knn_a_mano(X_ent, y_ent, x_nuevo, k):
    """La vía 1: distancia, ordenar, votar. Devuelve la proporción de votos
    (= la probabilidad de la clase 1)."""
    d = np.sqrt(((X_ent - x_nuevo) ** 2).sum(axis=1)) # distancia a todos
    vecinos = np.argsort(d)[:k] # los k más cercanos
    return y_ent[vecinos].mean() # proporción de votos

x_nuevo = sc_d.transform(nuevo)[0] # el cliente nuevo, escalado
p_mano = knn_a_mano(X_train_d_s, y_train_d, x_nuevo, K)
p_lib = modelo_d.predict_proba(sc_d.transform(nuevo))[0, 1]
print(f"a mano = {p_mano:.3f}")
print(f"sklearn = {p_lib:.3f}")
assert abs(p_mano - p_lib) < 1e-12
print("Idénticas: la librería hace exactamente la cuenta del dibujo.")
```

a mano = 0.667
sklearn = 0.667
Idénticas: la librería hace exactamente la cuenta del dibujo.

8. Predecir el conjunto de prueba

```
y_pred_d = modelo_d.predict(X_test_d_s)
y_pred_a = modelo_a.predict(X_test_a_s)

print("primeras 10 predicciones vs. realidad (determinista):")
print(np.concatenate((y_pred_d[:10].reshape(-1, 1),
                      y_test_d[:10].reshape(-1, 1)), axis=1))
```

primeras 10 predicciones vs. realidad (determinista):

```
[[1 1]
 [0 0]
 [1 0]
 [1 1]
 [0 0]
 [1 0]
 [0 0]
 [0 0]
 [1 1]
 [1 1]]
```

9. Las métricas

```
def tablero(y_real, y_pred, titulo):
    """El tablero del laboratorio: imprime la matriz de confusión y las
    métricas. La tasa base es la cifra que hay que superar."""
    # labels=[0,1] fija el orden de clases aunque una no aparezca en y_pred
    VN, FP, FN, VP = confusion_matrix(y_real, y_pred, labels=[0, 1]).ravel()
    base = max(y_real.mean(), 1 - y_real.mean())
    print(f"--- {titulo} ---")
    print(f"           predicho NO    predicho SÍ")
    print(f"real NO           {VN:5d}       {FP:5d}")
    print(f"real SÍ           {FN:5d}       {VP:5d}")
    print(f" tasa base : {base:.3f}    <- hay que superarla")
    print(f" exactitud : {accuracy_score(y_real, y_pred):.3f}")
    print(f" precisión : {precision_score(y_real, y_pred, pos_label=1, zero_division=0):.3f}")
    print(f" recall    : {recall_score(y_real, y_pred, pos_label=1, zero_division=0):.3f}")
    print(f" F1        : {f1_score(y_real, y_pred, pos_label=1, zero_division=0):.3f}\n")
    return dict(exactitud=accuracy_score(y_real, y_pred),
                precision=precision_score(y_real, y_pred, pos_label=1, zero_division=0),
                recall=recall_score(y_real, y_pred, pos_label=1, zero_division=0),
                F1=f1_score(y_real, y_pred, pos_label=1, zero_division=0), base=base)

m_d = tablero(y_test_d, y_pred_d, "PARTICIÓN DETERMINISTA (profesor)")
m_a = tablero(y_test_a, y_pred_a, f"PARTICIÓN ALEATORIA (semilla {MI_SEMILLA})")
```

```
--- PARTICIÓN DETERMINISTA (profesor) ---
           predicho NO    predicho SÍ
real NO           37       6
real SÍ           7       50
```

```
tasa base : 0.570    <- hay que superarla
exactitud : 0.870
precisión : 0.893
recall    : 0.877
F1        : 0.885
```

```
--- PARTICIÓN ALEATORIA (semilla 0) ---
```

```

                predicho NO    predicho SÍ
real NO          38          12
real SÍ           8          42
tasa base : 0.500    <- hay que superarla
exactitud : 0.800
precisión : 0.778
recall    : 0.840
F1        : 0.808
```

```

etq = ["exactitud", "precision", "recall", "F1"]
x = np.arange(len(etq)); ancho = 0.36
fig, ax = plt.subplots(figsize=(7, 3.2))
ax.bar(x - ancho/2, [m_d[e] for e in etq], ancho, label="determinista", color=AZUL)
ax.bar(x + ancho/2, [m_a[e] for e in etq], ancho, label="aleatoria", color=NARANJA)
ax.axhline(m_d["base"], ls=":", color=ROJO, lw=1.8, label="tasa base")
ax.set_xticks(x); ax.set_xticklabels(etq); ax.set_ylim(0, 1.05)
ax.legend(frameon=False, fontsize=8)
for i, e in enumerate(etq):
    ax.text(i - ancho/2, m_d[e] + .02, f"{m_d[e]:.2f}", ha="center", fontsize=7.5)
    ax.text(i + ancho/2, m_a[e] + .02, f"{m_a[e]:.2f}", ha="center", fontsize=7.5)
plt.tight_layout(); plt.show()
```

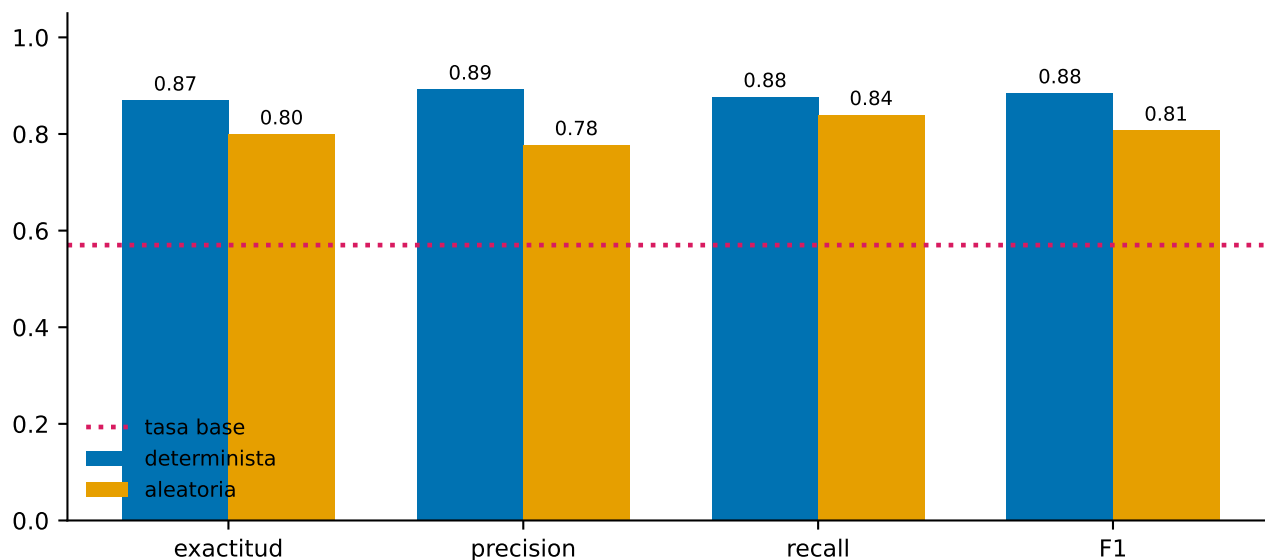


Figura 2: Las mismas métricas con las dos particiones. Parecidas, no idénticas.

La pregunta de la sesión: las dos fronteras se parecen pero **no son iguales**, y las métricas tampoco. ¿Está mal alguna de las dos? No: son dos muestras distintas del mismo problema. Lo que debe coincidir es la **conclusión**, no el tercer decimal.

10. Ver la frontera del modelo

```
def dibuja(ax, modelo, sc, X_set, y_set, titulo):
    """Dibuja la frontera de decisión en coordenadas ORIGINALES
    (el modelo vive en el mundo escalado; el dibujo, en el nuestro)."""
    X_set = sc.inverse_transform(X_set)      # volver de z-scores a años/millones
    X1, X2 = np.meshgrid(
        np.arange(X_set[:, 0].min() - 2, X_set[:, 0].max() + 2, 0.25),
        np.arange(X_set[:, 1].min() - 0.5, X_set[:, 1].max() + 0.5, 0.05))
    Z = modelo.predict(sc.transform(np.array([X1.ravel(), X2.ravel()]).T)).reshape(X1.shape)
    ax.contourf(X1, X2, Z, alpha=0.35, cmap=ListedColormap((ROJO, VERDE)))
    for cls, color in [(0, ROJO), (1, VERDE)]:
        ax.scatter(X_set[y_set == cls, 0], X_set[y_set == cls, 1],
                   c=color, s=14, edgecolor="white", linewidth=0.3)
    ax.set_title(titulo, fontsize=9.5)
    ax.set_xlabel("edad"); ax.set_ylabel("ingreso")

fig, axs = plt.subplots(1, 2, figsize=(11, 3.8))
dibuja(axs[0], modelo_d, sc_d, X_train_d_s, y_train_d, "determinista (entrenamiento)")
dibuja(axs[1], modelo_a, sc_a, X_train_a_s, y_train_a, f"aleatoria, semilla {MI_SEMILLA}")
plt.tight_layout(); plt.show()
```

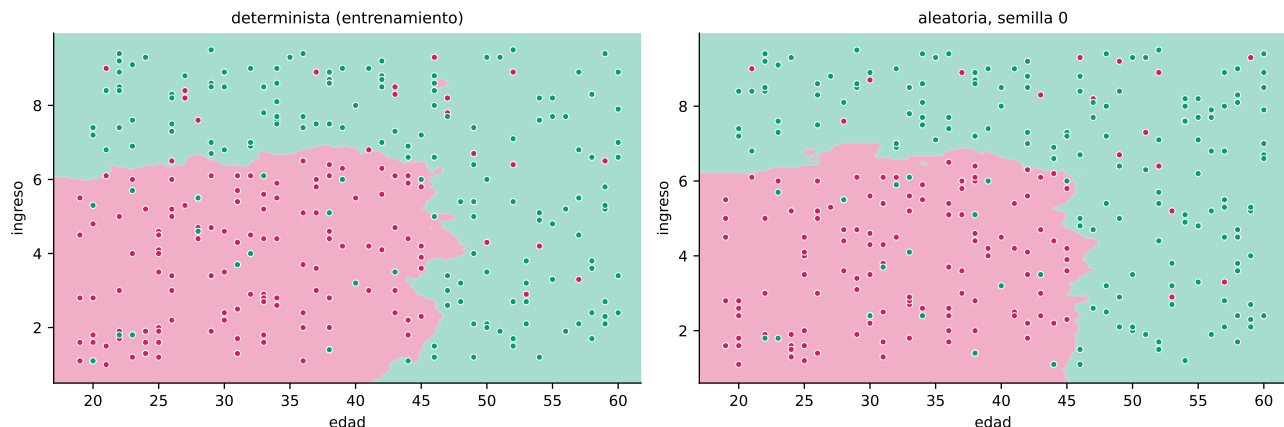


Figura 3: Qué decide el modelo en cada zona del plano, con las dos particiones.

11. K-NN a mano: el ejercicio que puede caer en el parcial

El profesor también hace K-NN **a mano** (la vía 1). Este es el formato típico: un historial pequeño, un cliente nuevo, k vecinos y votar.

```
# Tuplas: (cliente, edad, ingreso, tomo_credito)
# CAMBIA AQUÍ si el parcial trae otra tabla de clientes.
clientes = [("C001", 45, 3.9, 0), ("C002", 45, 7.2, 1),
            ("C003", 32, 6.9, 1), ("C004", 24, 1.6, 0),
            ("C005", 28, 7.6, 0), ("C006", 47, 8.2, 0),
            ("C007", 53, 2.1, 1), ("C008", 54, 4.2, 0),
            ("C009", 32, 7.0, 0), ("C010", 53, 2.9, 0)]
mini = pd.DataFrame(clientes, columns=["cliente", "edad", "ingreso", "tomo_credito"])
```

```

X = mini[["edad", "ingreso"]].values.astype(float)
y = mini["tomo_credito"].values.astype(int)

# El escalado del historial: media y desviación POBLACIONAL (la de sklearn)
media, desv = X.mean(axis=0), X.std(axis=0)
Xz = (X - media) / desv
print(f"media: edad={media[0]:.1f}, ingreso={media[1]:.2f} | "
      f"desv: edad={desv[0]:.2f}, ingreso={desv[1]:.2f}")
mini

```

media: edad=41.3, ingreso=5.16 | desv: edad=10.70, ingreso=2.35

	cliente	edad	ingreso	tomo_credito
0	C001	45	3.9	0
1	C002	45	7.2	1
2	C003	32	6.9	1
3	C004	24	1.6	0
4	C005	28	7.6	0
5	C006	47	8.2	0
6	C007	53	2.1	1
7	C008	54	4.2	0
8	C009	32	7.0	0
9	C010	53	2.9	0

11.1. Paso a paso con un solicitante

```

def distancias(X_usar, S):
    """Distancia euclídea del punto S a cada fila de X_usar (Pitágoras)."""
    return pd.Series(np.sqrt(((X_usar - S) ** 2).sum(axis=1)))

def votar_knn(X_usar, y_cli, S, k, nombres):
    """El K-NN del dibujo: distancias, los k más cercanos, votación.
    Empate exacto de votos (k par) -> clase 0 (lo que hace sklearn)."""
    d = distancias(X_usar, S)
    k_mas = d.sort_values(kind="stable").head(k)
    votos = y_cli[k_mas.index]
    clase = int(votos.sum() > k / 2) # mayoría ESTRICTA; empate -> 0
    t = pd.DataFrame({"cliente": nombres[k_mas.index],
                     "distancia": k_mas.values.round(3),
                     "clase_del_vecino": votos})
    return t, int(votos.sum()), k - int(votos.sum()), clase

```

```

S1 = np.array([40.0, 5.0]) # <<< CAMBIA AQUÍ el solicitante
S1z = (S1 - media) / desv

t_sin, v1s, v0s, c_sin = votar_knn(X, y, S1, 3, mini["cliente"].values)
t_con, v1c, v0c, c_con = votar_knn(Xz, y, S1z, 3, mini["cliente"].values)

print("SIN escalar (la edad manda: sus números son más grandes):")
print(t_sin.to_string(index=False))

```

```
print(f"votos: clase 1 = {v1s}, clase 0 = {v0s} -> predice {c_sin}\n")
print("CON escalar (las dos variables pesan parejo):")
print(t_con.to_string(index=False))
print(f"votos: clase 1 = {v1c}, clase 0 = {v0c} -> predice {c_con}")
```

SIN escalar (la edad manda: sus números son más grandes):

```
cliente  distancia  clase_del_vecino
C001      5.120           0
C002      5.463           1
C006      7.697           0
votos: clase 1 = 1, clase 0 = 2 -> predice 0
```

CON escalar (las dos variables pesan parejo):

```
cliente  distancia  clase_del_vecino
C001      0.661           0
C002      1.045           1
C003      1.100           1
votos: clase 1 = 2, clase 0 = 1 -> predice 1
```

La lección en una línea. Mismo solicitante, mismo $k=3$: sin escalar gana la clase **0** y con escalar la **1**. La distancia mezcla años y millones en una misma fórmula; sin escalar, la variable de números grandes (la edad) decide sola quién es vecino.

```
S2 = np.array([38.0, 8.5]) # <<< CAMBIA AQUÍ el segundo solicitante
S2z = (S2 - media) / desv
_, v1s2, v0s2, c_sin2 = votar_knn(X, y, S2, 3, mini["cliente"].values)
_, v1c2, v0c2, c_con2 = votar_knn(Xz, y, S2z, 3, mini["cliente"].values)
print(f"S2 = (38, 8.5), k=3")
print(f" SIN escalar: votos 1={v1s2}, 0={v0s2} -> clase {c_sin2}")
print(f" CON escalar: votos 1={v1c2}, 0={v0c2} -> clase {c_con2}")
```

```
S2 = (38, 8.5), k=3
SIN escalar: votos 1=2, 0=1 -> clase 1
CON escalar: votos 1=1, 0=2 -> clase 0
```

11.2. El efecto de k sobre la votación

```
filas_k = []
for k in [1, 3, 5, 7, 9]: # <<< CAMBIA AQUÍ los k a probar
    _, v1, v0, c = votar_knn(Xz, y, S1z, k, mini["cliente"].values)
    filas_k.append({"k": k, "votos clase 1": v1, "votos clase 0": v0,
                    "probabilidad (1/k)·Σy": v1 / k, "predicción": c})
display(tabla(pd.DataFrame(filas_k)))
```

Tabla 1: La votación de S1 (escalado) al crecer k .

k	votos clase 1	votos clase 0	probabilidad ($1/k$) · Σy	predicción
1	0	1	0.000	0
3	2	1	0.667	1

k	votos clase 1	votos clase 0	probabilidad (1/k) · Σy	predicción
5	2	3	0.400	0
7	2	5	0.286	0
9	3	6	0.333	0

12. El parámetro que lo cambia todo: k

12.1. De memorizar a aplanar

```
# k impares de 1 a 59; se entrena UNA vez por k (partición determinista).
ks = list(range(1, 60, 2))          # <<< CAMBIA AQUÍ el rango
acc_tr, acc_te = [], []
for k in ks:
    m = KNeighborsClassifier(n_neighbors=k).fit(X_train_d_s, y_train_d)
    acc_tr.append(m.score(X_train_d_s, y_train_d))
    acc_te.append(m.score(X_test_d_s, y_test_d))
base_te = max(y_test_d.mean(), 1 - y_test_d.mean())

fig, ax = plt.subplots(figsize=(7.5, 3.8))
ax.plot(ks, acc_tr, "-", color=GRIS, label="entrenamiento")
ax.plot(ks, acc_te, "o-", color=AZUL, label="prueba")
ax.axhline(base_te, ls=":", color=NARANJA, lw=1.8, label="tasa base")
ax.set_xlabel("k (n_neighbors)"); ax.set_ylabel("exactitud")
ax.legend(frameon=False, fontsize=8)
plt.tight_layout(); plt.show()
```

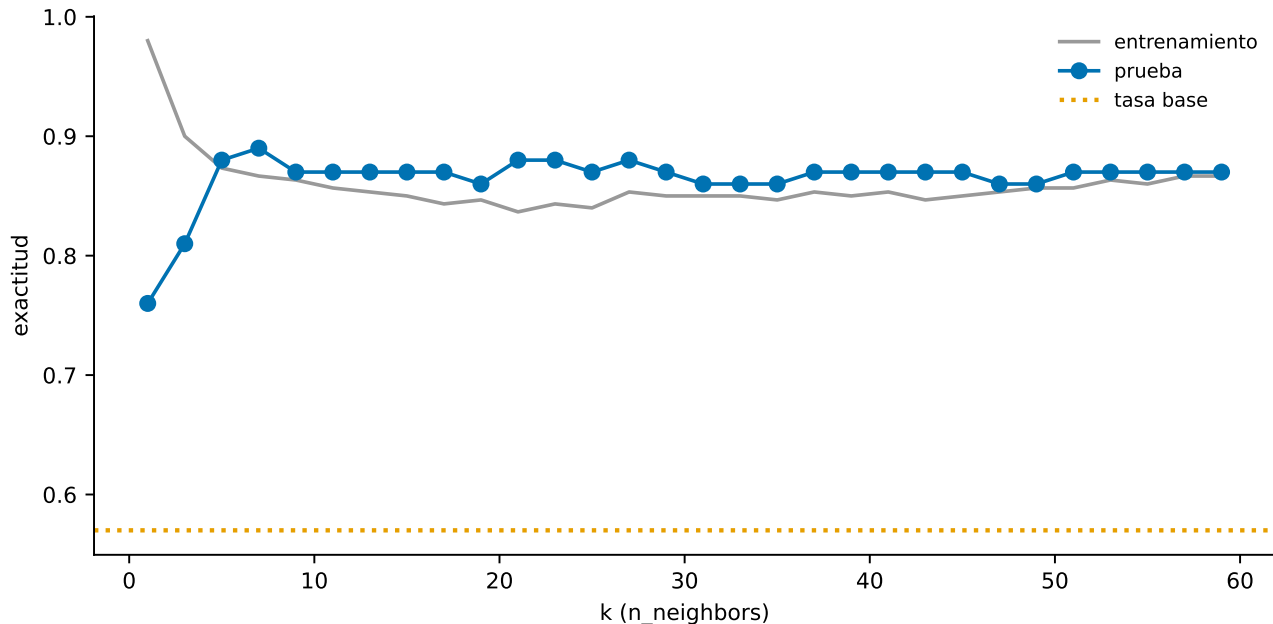


Figura 4: Exactitud en entrenamiento y prueba contra k . La de entrenamiento es perfecta en $k=1$ y engaña; la de prueba tiene un óptimo.

Reconozca la forma: es **la misma figura de la regularización** (donde el eje era α). El entrenamiento es perfecto en $k = 1$ (**0.980**) y solo empeora; la prueba sube, llega a un óptimo y cae. Aquí el mejor de la lista es $k = 7$ **con 0.890** (tasa base 0.570).

k es un **hiperparámetro**: algo que usted elige y el algoritmo no aprende. La forma elegante de elegirlo (validación cruzada) viene en el corte 3; por hoy, mirando la prueba. Y con $k = n$ (todos los clientes) la frontera desaparece: el modelo predice siempre la clase mayoritaria y su exactitud ES la tasa base — subajuste total.

13. La trampa de K-NN

13.1. Si no escala, una variable se come a la otra

```
a, b = X_train_d[0], X_train_d[1]      # dos clientes cualesquiera
print(f"cliente A: edad {a[0]:.0f}, ingreso {a[1]:.1f}")
print(f"cliente B: edad {b[0]:.0f}, ingreso {b[1]:.1f}")
print(f"aporte de la EDAD    a la distancia²: {(a[0]-b[0])**2:.4f}")
print(f"aporte del INGRESO  a la distancia²: {(a[1]-b[1])**2:.4f}")
```

```
cliente A: edad 45, ingreso 3.9
cliente B: edad 45, ingreso 7.2
aporte de la EDAD    a la distancia²: 0.0000
aporte del INGRESO  a la distancia²: 10.8900
```

Por qué pasa: en la fórmula de la distancia, una diferencia de 20 años de edad aporta $20^2 = 400$, mientras que una diferencia de 4 millones de ingreso aporta $4^2 = 16$. **La variable de números grandes decide quién es vecino**, aunque sea la menos informativa.

13.2. El experimento de la clase: la misma información, otras unidades

```
# EXACTAMENTE los mismos clientes; solo cambia la unidad del ingreso.
filas_u = []
for nombre, factor in [("millones", 1), ("miles", 1e3), ("pesos", 1e6)]:
    m = KNeighborsClassifier(K).fit(np.c_[X_train_d[:, 0],
                                         X_train_d[:, 1] * factor], y_train_d)
    ex = m.score(np.c_[X_test_d[:, 0], X_test_d[:, 1] * factor], y_test_d)
    filas_u.append({"ingreso en": nombre, "exactitud en prueba": ex})
display(tabla(pd.DataFrame(filas_u)))
```

ingreso en	exactitud en prueba
millones	0.860
miles	0.700
pesos	0.700

El dato no cambió: cambió la **unidad**. En pesos, el ingreso aplasta a la edad y la exactitud cae de **0.860 a 0.700**, casi la tasa base (0.570): el modelo dejó de aprender. **Escalar no es cosmética**.

13.3. Las dos debilidades que hay que decir en voz alta

1. **No hay entrenamiento, pero la predicción es cara.** Cada predicción calcula la distancia a todos los clientes guardados: $O(n \cdot p)$. Con millones de registros, K-NN es lento justo cuando más se necesita.
2. **La maldición de la dimensionalidad.** Con muchas variables, todos los puntos quedan lejos unos de otros y “vecino” pierde sentido.

```
rng = np.random.default_rng(42)          # la semilla de la clase
filas_m = []
for p in [2, 10, 50, 200]:                # <<< CAMBIA AQUÍ las dimensiones
    Z = rng.random((400, p))
    dd = np.sqrt(((Z[:200] - Z[200:]) ** 2).sum(axis=1))
    filas_m.append({"variables (p)": p, "distancia media": dd.mean(),
                    "razón cercano/lejano": dd.min() / dd.max()})
display(tabla(pd.DataFrame(filas_m)))
```

Tabla 2: Distancia media entre puntos aleatorios al aumentar la dimensión: con 200 variables, el más cercano queda casi tan lejos como el más lejano.

variables (p)	distancia media	razón cercano/lejano
2	0.519	0.014
10	1.270	0.373
50	2.893	0.674
200	5.743	0.815

Con 2 variables, el punto más cercano está claramente más cerca que el más lejano (razón pequeña). Con 200, todos quedan a distancias parecidas (razón cerca de 1): ya no hay “vecinos”.

14. Tres ideas para llevarse

1. **K-NN no ajusta nada:** guarda los datos y pregunta a los k parecidos; la proporción de votos **ya es una probabilidad**.
2. **k es un hiperparámetro con el dilema de siempre:** chico memoriza, grande aplana; se elige mirando la prueba, no el entrenamiento.
3. **Escalar no es opcional en K-NN:** la distancia la decide la variable de números grandes si no se interviene.

15. Resumen consolidado

Tabla 3: Cifras de esta corrida (partición determinista, $k=9$, escalado).

Cifra	Valor
Clientes (entrenamiento / prueba)	300 / 100
Proporción que tomó el crédito (cartera)	0.540
Tasa base de la prueba	0.570
k usado (impar, el del laboratorio)	9
Exactitud con ese k	0.870
Precisión / Recall / F1	0.893 / 0.877 / 0.885
Mejor k de la lista (por prueba, escalado)	7 (0.890)
Exactitud $k=1$ (memoriza)	0.760

Tabla 3: Cifras de esta corrida (partición determinista, $k=9$, escalado).

Cifra	Valor
Mi semilla (partición aleatoria)	0

16. Lo que usted entrega

i Instrucciones (las del laboratorio, más las de repaso)

1. Cambie `MI_SEMILLA` por los **dos últimos dígitos de su código**.
2. Renderice este documento a **PDF** (**quarto render**), con el código visible.
3. Al final del PDF, escriba a mano sus respuestas a estas preguntas:
 - a. ¿Su exactitud superó la **tasa base**? ¿Por cuánto?
 - b. Compare su tablero con el de la partición determinista: ¿cuál métrica cambió más y cuál menos?
 - c. Cambie K a 1 y vuelva a correr: ¿qué le pasó a la frontera del dibujo y a la exactitud? Explique en una frase.
 - d. En este problema, ¿qué error le preocuparía más al banco: un falso positivo (ofrecerle el crédito a quien no lo va a tomar) o un falso negativo (no ofrecérselo a quien sí lo tomaría)? Justifique.

De repaso, para el parcial:

- e. Con el cliente $S1 = (40, 5.0)$ del K-NN a mano: ¿por qué la predicción cambia al escalar, si el cliente es el mismo? (Use la palabra *distancia* en su respuesta.)
- f. Si le pidieran $p = 1$ en `KNeighborsClassifier`, ¿qué distancia estaría usando y qué nombre tiene?